A kicked one-dimensional system: densities, observables and TTCF with jumps

Working notes on Exercise 9.7.1.1

7 November 2026

Abstract

We take the one-dimensional system $\dot{x} = x^2 - 1$ on $[-1, 1]$ of Exercise 9.7.1.1 and add, at every time T, an instantaneous "fire" $x \mapsto x/K$, $K > 1$. The state space is not restricted: the fire only moves probability towards 0, and the support of the density shrinks by itself. We derive the evolution of the density, three equivalent representations of the expectation of an observable (direct/Lagrangian, Kawasaki/Eulerian, TTCF), and an exact transient time correlation function (TTCF) formula that includes the jumps. All maps are Möbius transformations, so everything can be checked against closed-form results. A short Python script (Appendix A) verifies the formulae numerically.

Contents

1 The model **2**
1.1 Continuous part . . . 2
1.2 The fire . . . 2
1.3 Evolution maps . . . 2

2 Densities **2**
2.1 Continuous evolution: Lagrangian and Eulerian forms . . . 2
2.2 The jump . . . 3
2.3 Composition . . . 3
2.4 The support shrinks by itself . . . 4

3 Expectations of observables **4**
3.1 Direct (Lagrangian, Heisenberg-like) . . . 4
3.2 Kawasaki (Eulerian): the "new observable" picture . . . 5
3.3 TTCF with jumps . . . 5

4 TTCF with jumps **5**
4.1 Reminder: the autonomous case . . . 5
4.2 Derivation with jumps . . . 5
4.3 Using periodicity: correlation functions to compute . . . 6
4.4 Implementation . . . 7

5 Closed-form results for this toy model **7**
5.1 Mean of x . . . 7
5.2 Fixed point and asymptotic state . . . 7
5.3 Numerical check . . . 8

6 Remarks and open directions **8**

A Python verification script 8

1 The model

1.1 Continuous part

On the phase space $\mathcal{M} = [-1, 1]$ consider

$$\dot{x} = G(x) = x^2 - 1, \qquad \Lambda(x) = G'(x) = 2x. \tag{1}$$

The endpoints ± 1 are fixed points, $\mathcal{M}$ is invariant forwards and backwards in time, -1 is attracting and $+1$ repelling. Since $\dot{x} = -(1 - x^2)$, the solution is $x(t) = \tanh(\operatorname{artanh} x_0 - t)$, i.e., with $\tau_t = \tanh t$,

$$\Phi^t x = \frac{x - \tau_t}{1 - \tau_t x} = \frac{(1 + x) - (1 - x)e^{2t}}{(1 + x) + (1 - x)e^{2t}}, \tag{2}$$

which coincides with Eq. (9.131) of the book and is valid for all $x \in \mathcal{M}$, including $x = \pm 1$. Its derivative is

$$\frac{\mathrm{d}\Phi^t x}{\mathrm{d}x} = \frac{1 - \tau_t^2}{(1 - \tau_t x)^2} = \exp\Big(\int_0^t \Lambda(\Phi^s x)\, \mathrm{d}s \Big). \tag{3}$$

1.2 The fire

At the times $T, 2T, 3T, \ldots$ the state jumps according to

$$\psi(x) = x/K, \qquad K > 1. \tag{4}$$

ψ maps $\mathcal{M}$ *into* $\mathcal{M}$, onto the smaller interval $\psi(\mathcal{M}) = [-1/K, 1/K]$. We do *not* change the phase space: after a fire the flow (2) continues to act on the whole of $\mathcal{M}$. Physically: before the fire many trees are likely; after it, states above $1/K$ cannot be reached, and probability is moved towards 0 while keeping its shape.

1.3 Evolution maps

Let $S_{s,t}$ denote the evolution from time s to time $t \geq s$, with the convention that a jump at time kT is applied when the trajectory reaches kT; a trajectory that *starts* at $s = kT$ starts just after the k-th fire. We write $S_t = S_{0,t}$. The one-period map and the evolution up to $t = nT + \tau$, $0 \leq \tau < T$, are

$$S = \psi \circ \Phi^T, \qquad S_t = \Phi^\tau \circ S^n. \tag{5}$$

The dynamics is T-periodic: $S_{s+T,t+T} = S_{s,t}$.

Remark 1 (Möbius structure). A map $x \mapsto (ax + b)/(cx + d)$ is represented by the matrix $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$, and composition corresponds to matrix multiplication. Here

$$\Phi^t \leftrightarrow \begin{pmatrix} 1 & -\tau_t \\ -\tau_t & 1 \end{pmatrix}, \qquad \psi \leftrightarrow \begin{pmatrix} 1 & 0 \\ 0 & K \end{pmatrix}, \qquad S \leftrightarrow \begin{pmatrix} 1 & -\tau_T \\ -K\tau_T & K \end{pmatrix}. \tag{6}$$

So S_t is a Möbius map for every t, and its matrix is a product of 2×2 matrices.

2 Densities

2.1 Continuous evolution: Lagrangian and Eulerian forms

Let $\Lambda_{a,b}(x) = \int_a^b \Lambda(\Phi^s x)\, \mathrm{d}s$. Under the flow,

$$f_t(x) = e^{-\Lambda_{-t,0}(x)}\, f_0(\Phi^{-t} x) \qquad \text{(Lagrangian, Eq. (9.141))} \tag{7}$$

$$= e^{\Omega^{(0)}_{-t,0}(x)}\, f_0(x) \qquad \text{(Eulerian, Eq. (9.142))} \tag{8}$$

where the dissipation function of a density f is

$$\Omega^{(f)}(x) = -\Lambda(x) - G(x)\,\frac{f'(x)}{f(x)}, \qquad f\,\Omega^{(f)} = -\frac{\mathrm{d}}{\mathrm{d}x}\big(Gf\big), \tag{9}$$

and $\Omega^{(0)} = \Omega^{(f_0)}$. Integrating along the trajectory,

$$\Omega^{(0)}_{-t,0}(x) = -\Lambda_{-t,0}(x) + \ln f_0(\Phi^{-t}x) - \ln f_0(x), \tag{10}$$

so (7) and (8) coincide: in the Eulerian form f_0 is evaluated at x, and the back-evolution is hidden inside $\Omega^{(0)}$.

For $f_0 = 1/2$ one has $\Omega^{(0)} = -\Lambda = -2x$ (Eq. (9.139)) and

$$f_t(x) = R_t(x)\,f_0(x), \qquad R_t(x) = e^{-\Lambda_{-t,0}(x)} = \frac{\mathrm{d}\Phi^{-t}x}{\mathrm{d}x} = \frac{1-\tau_t^2}{(1+\tau_t x)^2} = \frac{4e^{-2t}}{\big[(1+x)+(1-x)e^{-2t}\big]^2}. \tag{11}$$

Remark 2 (A misprint in the book). In Eq. (9.137) the factor 2 of $\Lambda = 2x$ is missing: the correct result is $\int_0^t \Lambda(\Phi^s x)\,\mathrm{d}s = 2t - 2\ln\frac{A+Be^{2t}}{2}$, $A = 1+x$, $B = 1-x$. Consequently Eq. (9.138) should read $f^{(t)}(x) = \frac{1}{2}\cdot 4e^{-2t}/(A+Be^{-2t})^2$, which is (11). One checks that $\int_{-1}^{1} R_t(x)\,\frac{1}{2}\,\mathrm{d}x = 1$, whereas the expression printed in (9.138) integrates to $2te^{-t}/(1-e^{-2t}) \neq 1$. Also, below (9.139), "$f_0$ is constant in $[0,1]$" should read $[-1,1]$.

2.2 The jump

The density is transported by ψ (Frobenius–Perron):

$$f_{kT^+}(x) = K\,f_{kT^-}(Kx)\,\mathbf{1}_{\psi(\mathcal{M})}(x), \qquad \psi(\mathcal{M}) = [-1/K, 1/K]. \tag{12}$$

Both the factor K (the Jacobian of ψ^{-1}) and the argument Kx are needed. With them, normalization is automatic, $\int f_{kT^+} = \int f_{kT^-} = 1$, and the shape is preserved: f_{kT^+} is f_{kT^-} squeezed into $[-1/K, 1/K]$. Simply multiplying by K and cutting, $K f_{kT^-}(x)\mathbf{1}_{\psi(\mathcal{M})}(x)$, would *not* conserve probability. It would correspond to removing whole realizations from the ensemble, which is a different problem (escape, conditioning).

2.3 Composition

For any t,

$$f_t(x) = f_0\big(S_t^{-1}x\big)\,J_t(x)\,\mathbf{1}_{S_t(\mathcal{M})}(x), \qquad J_t(x) = \Big|\frac{\mathrm{d}S_t^{-1}x}{\mathrm{d}x}\Big|. \tag{13}$$

Equivalently, by recursion over one period,

$$f_{nT^+}(x) = K\,R_T(Kx)\,f_{(n-1)T^+}\big(\Phi^{-T}(Kx)\big)\,\mathbf{1}_{[-1/K,1/K]}(x). \tag{14}$$

Each step of (14) contains exactly one jump, hence one factor K and the indicator of $\psi(\mathcal{M}) = [-1/K, 1/K]$, whatever the number of the jump. The earlier jumps are hidden in $f_{(n-1)T^+}$. Unrolling the recursion, with $y_n = x$ and $y_{j-1} = \Phi^{-T}(Ky_j)$,

$$f_{nT^+}(x) = K^n \prod_{j=1}^{n} R_T(Ky_j)\; f_0(y_0)\; \prod_{j=1}^{n} \mathbf{1}_{[-1/K,1/K]}(y_j), \qquad \prod_{j=1}^{n} \mathbf{1}_{[-1/K,1/K]}(y_j) = \mathbf{1}_{S^n(\mathcal{M})}(x). \tag{15}$$

The factor K^n appears explicitly, and the product of indicators is the indicator of the true support $S^n(\mathcal{M})$ in (13). That support is *not* $[-K^{-n}, K^{-n}]$, which would be correct only if the flow were rescaled after each fire. For example, for $K = 2$, $T = 0.5$, after two jumps $S^2(\mathcal{M}) = [-0.3908,\ 0.0246]$ (Table 1), while $[-K^{-2}, K^{-2}] = [-0.25, 0.25]$.

Remark 3 (Composition of the factors). The factor from 0 to $2T$ is *not* $R_T(x)^2$. By the cocycle property, $R_T(x)\,R_T(\Phi^{-T}x) = R_{2T}(x)$. In the Eulerian language: for the second interval the initial density is f_T, which is not uniform, so the relevant dissipation function is $\Omega^{(f_T)} \neq \Omega^{(f_0)}$. Multiplying f_T by a constant (such as K) does not change $\Omega^{(f_T)}$, because only $\mathrm{d}\ln f/\mathrm{d}x$ enters (9).

2.4 The support shrinks by itself

Since $\Phi^T(\mathcal{M}) = \mathcal{M}$, we get $S(\mathcal{M}) = \psi(\mathcal{M}) = [-1/K, 1/K]$ and, by induction, $S^n(\mathcal{M}) = [l_n, u_n]$ with

$$l_n = S(l_{n-1}), \quad u_n = S(u_{n-1}), \qquad l_0 = -1,\ u_0 = 1. \tag{16}$$

Since $\Phi^T u < u$ for $u \in (-1, 1)$, we get $u_n < u_{n-1}/K$, so $u_n < K^{-n}$ for $n \geq 2$: the maximum number of trees never recovers. The intervals are nested, $S^n(\mathcal{M}) \subset S^{n-1}(\mathcal{M})$, and both endpoints converge to the fixed point x^* of S (Section 5). The shrinking need not be imposed; it is a consequence of the dynamics.

The support is not $[-K^{-n}, K^{-n}]$. At each period the endpoints are mapped by the full one-period map $S = \psi \circ \Phi^T$: first the flow moves them, then the fire divides them by K,

$$l_n = \Phi^T(l_{n-1})/K, \qquad u_n = \Phi^T(u_{n-1})/K. \tag{17}$$

Dividing only by K would give $[-K^{-n}, K^{-n}]$. The flow pushes every interior point down towards -1 ($\Phi^T u < u$), so for $n \geq 2$

- the upper edge moves *inside*: $u_n < K^{-n}$;
- the lower edge moves *outside*: $l_n < -K^{-n}$. These are points that the continuous evolution carries out of $[-K^{-n}, K^{-n}]$.

For $K = 2$, $T = 0.5$, at $n = 2$ the flow first takes -0.5 to $\Phi^{0.5}(-0.5) \approx -0.7815$, and the fire then halves it to -0.3908:

n	$[l_n, u_n]$	$[-K^{-n}, K^{-n}]$
1	$[-0.5000,\ 0.5000]$	$[-0.5000,\ 0.5000]$
2	$[-0.3908,\ 0.0246]$	$[-0.2500,\ 0.2500]$
3	$[-0.3612,\ -0.2213]$	$[-0.1250,\ 0.1250]$
4	$[-0.3528,\ -0.3100]$	$[-0.0625,\ 0.0625]$
5	$[-0.3503,\ -0.3377]$	$[-0.0313,\ 0.0313]$

The two sequences of intervals also end up in different places: $[-K^{-n}, K^{-n}]$ shrinks to the point 0, whereas $[l_n, u_n]$ shrinks to the fixed point $x^* \approx -0.3493$ of S. This is why $\mathbb{E}_{nT^+}[x] \to x^*$, and not to 0.

3 Expectations of observables

Let O be any (bounded, measurable) function on $\mathcal{M}$; $O(x) = x$ is the simplest case. Three exact and equivalent representations are available.

3.1 Direct (Lagrangian, Heisenberg-like)

$$\mathbb{E}_t[O] = \int_{\mathcal{M}} f_t\, O\, \mathrm{d}x = \int_{\mathcal{M}} f_0(z)\, O(S_t z)\, \mathrm{d}z = \mathbb{E}_0[O \circ S_t]. \tag{18}$$

Sample initial points from f_0, evolve them with flow and jumps, and average O. No exponential factor appears: it is absorbed by the change of variables $x = S_t z$, whose Jacobian is exactly J_t.

3.2 Kawasaki (Eulerian): the "new observable" picture

Define, for $x \in S_t(\mathcal{M})$,

$$\Omega_t(x) = \ln \frac{f_0(S_t^{-1}x)\, J_t(x)}{f_0(x)}. \tag{19}$$

Then $f_t = f_0\, e^{\Omega_t}\, \mathbf{1}_{S_t(\mathcal{M})}$ and

$$\mathbb{E}_t[O] = \mathbb{E}_0\big[\, O\, e^{\Omega_t}\, \mathbf{1}_{S_t(\mathcal{M})}\big]. \tag{20}$$

$\ln J_t$ is the sum of $-\Lambda$ integrated over all flow segments of the backward trajectory, plus $n \ln K$ for n jumps. For $f_0 = 1/2$ at $t = nT^+$:

$$e^{\Omega_{nT}(x)} = K^n \prod_{j=1}^{n} R_T(Ky_j), \qquad y_n = x, \quad y_{j-1} = \Phi^{-T}(Ky_j). \tag{21}$$

This is the corrected version of the idea "new observable $O_n = O \times$ (exponential factor) $\times K^n$, same density f_0". The factor K^n is indeed there. The exponential factor is R_T evaluated at the back-mapped points, not $R_T(x)^n$, and the indicator of the support is essential.

3.3 TTCF with jumps

This is derived in the next section; see Eq. (27).

4 TTCF with jumps

4.1 Reminder: the autonomous case

For the flow alone, $\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}_0[O\circ\Phi^t] = \mathbb{E}_0[G\,(O\circ\Phi^t)']$ because $\frac{\mathrm{d}}{\mathrm{d}t}(O\circ\Phi^t) = G\,(O\circ\Phi^t)'$ (the generator commutes with the flow). Integrating by parts with (9),

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}_t[O] = \mathbb{E}_0\big[(O\circ\Phi^t)\,\Omega^{(0)}\big] + \Big[f_0\, G\,(O\circ\Phi^t)\Big]_{-1}^{1}. \tag{22}$$

The boundary term vanishes because $G(\pm 1) = 0$. Integration gives Eq. (9.144):

$$\mathbb{E}_t[O] = \mathbb{E}_0[O] + \int_0^t \mathrm{d}s\; \mathbb{E}_0\big[(O\circ\Phi^s)\,\Omega^{(0)}\big]. \tag{23}$$

With jumps the argument fails: $S_t = \Phi^\tau \circ S^n$ is not a semigroup in t, and the time derivative cannot be moved to time 0.

4.2 Derivation with jumps

Fix t and consider the expectation at time t of a system prepared in f_0 at a *variable* starting time $s \in [0,t]$:

$$F(s) = \mathbb{E}_0\big[O\circ S_{s,t}\big] = \int_{\mathcal{M}} f_0(x)\, O(S_{s,t}x)\,\mathrm{d}x. \tag{24}$$

Then $F(0) = \mathbb{E}_t[O]$ and $F(t) = \mathbb{E}_0[O]$.

- *Between jumps* ($s \neq kT$). The backward equation gives $\partial_s(O\circ S_{s,t})(x) = -G(x)\,(O\circ S_{s,t})'(x)$. Integrating by parts, the boundary term vanishes again since $G(\pm 1) = 0$, and

$$-F'(s) = \mathbb{E}_0\big[\Omega^{(0)}\;(O\circ S_{s,t})\big]. \tag{25}$$

- *At a jump* $s = kT \le t$. F is discontinuous, since $S_{kT^-,t} = S_{kT^+,t}\circ\psi$:

$$F(kT^-) - F(kT^+) = \mathbb{E}_0\big[O\circ S_{kT,t}\circ\psi - O\circ S_{kT,t}\big]. \tag{26}$$

Writing $F(0) - F(t) = -\int F' \, \mathrm{d}s + \sum_k \big[F(kT^-) - F(kT^+)\big]$, we obtain:

Proposition 1 (TTCF with jumps)**.** *For $nT \le t < (n+1)T$,*

$$\boxed{\mathbb{E}_t[O] = \mathbb{E}_0[O] + \int_0^t \mathrm{d}s \; \mathbb{E}_0\big[\Omega^{(0)} \, (O \circ S_{s,t})\big] + \sum_{k=1}^{n} \mathbb{E}_0\big[O \circ S_{kT,t} \circ \psi - O \circ S_{kT,t}\big]} \tag{27}$$

where $\Omega^{(0)} = -\Lambda - G f_0'/f_0$ is the dissipation function of the continuous *dynamics with respect to f_0, the s-integral runs over the flow intervals (the jump instants have zero measure), and $S_{s,t}$ includes all jumps between s and t.*

For $K = 1$ (no jumps) the sum vanishes, $S_{s,t} = \Phi^{t-s}$, and the change of variables $u = t - s$ gives back Eq. (9.144).

Interpretation.

- The first correction is a TTCF in the usual sense: $\Omega^{(0)}$ is evaluated at the initial point, drawn from f_0, and correlated with O at the end of a trajectory that *starts at time s*. Since the dynamics is not autonomous, the start time matters, and it is no longer a single correlation function of the lag $t - s$.
- Each jump contributes a finite "jump response": the difference of O at time t between a trajectory that starts at kT from $\psi(x)$ and one that starts at kT from x, averaged over $x \sim f_0$. It requires no dissipation function.
- Only f_0 and $\Omega^{(0)}$ ever appear. One never needs the dissipation function of a density with restricted support, and points whose backward trajectory leaves $S_t(\mathcal{M})$ cause no problems.

Remark 4 (Jump as a fast flow; boundary terms). ψ is the time-$\ln K$ map of the flow $\dot{x} = g(x) = -x$. Writing the jump term as $\int_0^{\ln K} \mathrm{d}\sigma \, \frac{\mathrm{d}}{\mathrm{d}\sigma} \mathbb{E}_0[A \circ \psi_\sigma]$, with $A = O \circ S_{kT,t}$ and $\psi_\sigma x = x e^{-\sigma}$, the same integration by parts gives a "kick dissipation function" $\Omega_{\mathrm{kick}} = -(g f_0)'/f_0 = 1 + x f_0'/f_0$. However, now $g(\pm 1) \neq 0$, so

$$\frac{\mathrm{d}}{\mathrm{d}\sigma} \mathbb{E}_0[A \circ \psi_\sigma] = \mathbb{E}_0\big[\Omega_{\mathrm{kick}} \, (A \circ \psi_\sigma)\big] - f_0(1) A(\psi_\sigma 1) - f_0(-1) A(\psi_\sigma(-1)). \tag{28}$$

The boundary terms vanish only if $f_0(\pm 1) = 0$, e.g. for $f_0 = \frac{3}{4}(1 - x^2)$, but not for $f_0 = \frac{1}{2}$. The finite form used in (27) avoids this subtlety altogether.

4.3 Using periodicity: correlation functions to compute

Since $S_{s+T,t+T} = S_{s,t}$, write $s = mT + \sigma$ with $\sigma \in (0, T)$, and define

$$C(\sigma, u) = \mathbb{E}_0\big[\Omega^{(0)}(x) \; O(S_{\sigma,\sigma+u} x)\big], \qquad \sigma \in (0,T), \; u \ge 0, \tag{29}$$

$$D(u) = \mathbb{E}_0\big[O(S_{0,u}(\psi x)) - O(S_{0,u} x)\big], \qquad u \ge 0, \tag{30}$$

where in D the trajectories start just after a jump (phase 0^+, next jump at T). Then, for $nT \le t < (n+1)T$,

$$\mathbb{E}_t[O] = \mathbb{E}_0[O] + \sum_{m=0}^{n} \int_0^{\min(T,\, t-mT)} \mathrm{d}\sigma \; C(\sigma, \, t - mT - \sigma) + \sum_{k=1}^{n} D(t - kT). \tag{31}$$

4.4 Implementation

1. **Initial ensemble.** Draw N points $x_i \sim f_0$ (or use quadrature nodes and weights). Compute $\Omega^{(0)}(x_i)$ once.
2. **Phase grid.** Choose $\sigma_1, \dots, \sigma_P \in (0, T)$ (e.g. Gauss–Legendre nodes).
3. **Flow correlations.** For each σ_p, evolve every x_i with the kicked dynamics starting at phase σ_p (first jump after a time $T - \sigma_p$), recording O on a grid of lags u. Accumulate $C(\sigma_p, u) \approx \frac{1}{N} \sum_i \Omega^{(0)}(x_i)\, O(\cdot)$.
4. **Jump correlations.** Evolve the pairs x_i and $\psi(x_i)$ from phase 0^+ and accumulate the difference of O to obtain $D(u)$.
5. **Assemble** with (31), using a quadrature in σ.

Cost. In the autonomous case one batch of trajectories gives the whole response. Here one needs one batch per starting phase σ_p, plus one batch of pairs for D. In exchange, as in standard TTCF, the correlations start from zero-mean quantities ($\mathbb{E}_0[\Omega^{(0)}] = 0$ by (9) with $G(\pm 1) = 0$; D is a difference), which is what gives TTCF its favourable signal-to-noise ratio when the perturbation is weak (here: K close to 1).

Pseudo-code.

```
x[i] ~ f0, i=1..N ; W[i] = Omega0(x[i])
for p in 1..P: # starting phases sigma_p in (0,T)
    y = x ; time_to_jump = T - sigma_p
    for each lag u on the grid:
        advance y by the flow; apply psi whenever a jump time is crossed
        C[p,u] += mean(W * O(y))
y1 = x ; y2 = psi(x) # pairs started at phase 0+
for each lag u on the grid:
    advance both (flow + jumps at u = T, 2T, ...)
    D[u] = mean(O(y2) - O(y1))
E_t[O] = E0[O] + sum_m int dsigma C(sigma, t-mT-sigma) + sum_k D(t-kT)
```

5 Closed-form results for this toy model

5.1 Mean of x

If $S_t \leftrightarrow \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ with $c \neq 0$, then for $f_0 = \frac{1}{2}$

$$\mathbb{E}_t[x] = \frac{1}{2} \int_{-1}^{1} \frac{az + b}{cz + d}\, \mathrm{d}z = \frac{a}{c} + \frac{bc - ad}{2c^2}\, \ln \left| \frac{d + c}{d - c} \right|. \tag{32}$$

5.2 Fixed point and asymptotic state

With $\tau = \tanh T$, the fixed point of S solves $x - \tau = Kx(1 - \tau x)$, i.e. $K\tau x^2 + (1 - K)x - \tau = 0$. The unique root in $\mathcal{M}$ is

$$x^* = \frac{(K - 1) - \sqrt{(K - 1)^2 + 4K\tau^2}}{2K\tau} \in (-1/K, 0). \tag{33}$$

Since

$$S'(x) = \frac{1 - \tau^2}{K(1 - \tau x)^2}, \qquad 0 < S'(x^*) < \frac{1 - \tau^2}{K} < 1, \tag{34}$$

x^* is attracting. S is increasing, so every orbit converges monotonically to x^*. The density converges to $\delta(x - x^*)$ at the times nT^+, and to $\delta(x - \Phi^\tau x^*)$ at the phase τ: the system

reaches a time-periodic asymptotic state. Unlike the unkicked system, where $\mathbb{E}_t[x] \to -1$, here $\mathbb{E}_{nT^+}[x] \to x^*$. The phase-space contraction per period at x^* is $\ln S'(x^*) = \Lambda_{0,T}(x^*) - \ln K < 0$: the flow contributes $\Lambda_{0,T}$, and each fire contributes $-\ln K$.

5.3 Numerical check

Table 1 compares (32) with the direct average (18), the TTCF formula (27), and the Kawasaki form (20), for $K = 2$, $T = 0.5$, $O(x) = x$, $f_0 = \frac{1}{2}$. The script also checks (27) for $f_0 = \frac{3}{4}(1 - x^2)$, where $\Omega^{(0)} = -4x$; direct and TTCF agree to all printed digits.

t	exact	direct	TTCF + jumps	Kawasaki	support $S_t(\mathcal{M})$
0.25	−0.1652900760	−0.1652900760	−0.1652900760	−0.1652900760	$[-1, 1]$
0.50	−0.1613031127	−0.1613031127	−0.1613031127	−0.1613031127	$[-0.5000, 0.5000]$
0.75	−0.3749729624	−0.3749729624	−0.3749729624	−0.3749729624	$[-0.6636, 0.2907]$
1.00	−0.2780722033	−0.2780722033	−0.2780722033	−0.2780722033	$[-0.3908, 0.0246]$
1.50	−0.3265087657	−0.3265087657	−0.3265087657	−0.3265087657	$[-0.3612, -0.2213]$
2.00	−0.3424778729	−0.3424778729	−0.3424778729	−0.3424778729	$[-0.3528, -0.3100]$
3.00	−0.3487418470	−0.3487418470	−0.3487418470	−0.3487418470	$[-0.3496, -0.3459]$
5.00	−0.3493266560	−0.3493266560	−0.3493266560	−0.3493266560	$[-0.3493, -0.3493]$

Table 1: $K = 2$, $T = 0.5$, $x^* = -0.3493309143$. The jumps occur at $t = 0.5, 1, 1.5, \ldots$; values at those times are taken just after the jump.

6 Remarks and open directions

- **Irreversibility.** ψ is injective but not onto $\mathcal{M}$, so the kicked dynamics is not invertible on $\mathcal{M}$, and there is no involution i with $iSi = S^{-1}$. Moreover, the support of f_t is strictly smaller than that of f_0, so the ergodic consistency condition is violated. Standard transient fluctuation relations for Ω_t should therefore not be expected. The response formula (27) uses neither reversibility nor ergodic consistency.

- **Other jump maps.** Nothing in the derivation of (27) uses the specific form of ψ: any measurable $\psi : \mathcal{M} \to \mathcal{M}$ applied at known times gives the same structure. Random fire intensities (K random) can be handled by averaging the jump terms over K; jumps at random times (e.g. Poisson) by averaging over the jump times. These have not been derived in detail here.

- **Towards a population model.** If x is the population size, the continuous part should contain the growth law (with births). The constraint "never back to the initial value" is a property of the combined flow-plus-jump dynamics, as in the toy model, where $\sup S^n(\mathcal{M}) < K^{-n}$. A flow with $G(\pm 1) \neq 0$ at the boundaries of the chosen phase space would add boundary terms to the TTCF integration by parts, which must be checked case by case.

- **Earlier reading, set aside.** Cutting the density and multiplying by K (removing realizations) does not conserve probability. It belongs to the escape/conditioning setting (survival probabilities), not to the present one.

A Python verification script

Requires only `numpy`. Run with `python3 kicked_ttcf.py`.

```
"""
Kicked one-dimensional system: dx/dt = x^2 - 1 on M = [-1, 1],
plus a "fire" x -> x/K applied at times T, 2T, 3T, ...

Three independent ways of computing E_t[O] starting from f0 = 1/2:
  1. exact : closed form, using the fact that all maps are Moebius maps
  2. direct : E_t[O] = E_0[O o S_t] (average over initial points)
  3. ttcf : E_0[O] + flow TTCF terms + jump terms (formula of the notes)
  4. kawasaki : E_0[O * exp(Omega_t) * 1_{S_t(M)}] (Eulerian / weighted f0)

Only numpy is needed. Run: python3 kicked_ttcf.py
"""
import numpy as np

K = 2.0 # fire factor
T = 0.5 # time between fires
O = lambda x: x # observable

# ------------------------------------------------------------------ dynamics
def flow(x, t):
    """Phi^t x = tanh(artanh(x) - t), written as a Moebius map (safe at x=+-1)."""
    th = np.tanh(t)
    return (x - th) / (1.0 - th * x)

def kick(x):
    return x / K

def evolve(x, s, t):
    """S_{s,t}: evolve from time s to time t >= s. Kicks at kT (k>=1), applied
    when the trajectory reaches kT; a start at s = kT means 'just after' kick k."""
    x = np.array(x, dtype=float)
    k_next = np.floor(s / T + 1e-12) + 1 # first kick strictly after s
    cur = s
    while k_next * T <= t + 1e-12:
        x = kick(flow(x, k_next * T - cur))
        cur = k_next * T
        k_next += 1
    return flow(x, t - cur)

# Gauss-Legendre nodes on [-1,1] for the average over f0 = 1/2
xg, wg = np.polynomial.legendre.leggauss(400)
E0 = lambda vals: 0.5 * np.sum(wg * vals)

# ------------------------------------------------------------------ 1. exact
def moebius_matrix(t):
    """2x2 matrix of S_{0,t} acting as x -> (a x + b)/(c x + d)."""
    def Mflow(tau):
        th = np.tanh(tau)
        return np.array([[1.0, -th], [-th, 1.0]])
    Mk = np.array([[1.0, 0.0], [0.0, K]])
    n = int(np.floor(t / T + 1e-12))
    M = np.eye(2)
    for _ in range(n):
        M = Mk @ Mflow(T) @ M
    return Mflow(t - n * T) @ M

def exact_mean_x(t):
    (a, b), (c, d) = moebius_matrix(t)
    if abs(c) < 1e-14: # t = 0: identity map
        return b / d
    # (1/2) int_{-1}^{1} (a x + b)/(c x + d) dx
    return 0.5 * (2 * a / c + (b * c - a * d) / c**2 * np.log(abs((c + d) / (d - c))))
```

```
# ------------------------------------------------------------------ 2. direct
def direct(t):
    return E0(O(evolve(xg, 0.0, t)))

# ------------------------------------------------------------------ 3. TTCF with jumps
Omega0 = lambda x: -2.0 * x # Omega^{(0)} = -Lambda - G f0'/f0, f0 uniform
sg, swg = np.polynomial.legendre.leggauss(200) # quadrature in s on each segment

def ttcf(t):
    n = int(np.floor(t / T + 1e-12))
    res = E0(O(xg))
    # flow part: sum over segments (kT, min((k+1)T, t))
    for k in range(n + 1):
        a, b = k * T, min((k + 1) * T, t)
        if b - a < 1e-14:
            continue
        s_nodes = 0.5 * (b - a) * sg + 0.5 * (a + b)
        inner = np.array([E0(Omega0(xg) * O(evolve(xg, s, t))) for s in s_nodes])
        res += 0.5 * (b - a) * np.sum(swg * inner)
    # jump part: sum over kicks k = 1..n
    for k in range(1, n + 1):
        res += E0(O(evolve(kick(xg), k * T, t)) - O(evolve(xg, k * T, t)))
    return res

# ------------------------------------------------------------------ 4. Kawasaki
def inverse_and_jacobian(t):
    """Matrix of S_t^{-1} and its determinant, for f_t = f0(S^-1 x)|dS^-1/dx| 1_{S_t(M)}."""
    M = moebius_matrix(t)
    Minv = np.linalg.inv(M)
    return Minv, np.linalg.det(Minv)

def kawasaki(t):
    """E_t[O] = int f0(x) exp(Omega_t(x)) 1_{S_t(M)}(x) O(x) dx with f0 = 1/2,
    where exp(Omega_t(x)) = |d S_t^{-1} x/dx| (Jacobian of the inverse map).
    The indicator is handled by integrating directly over the support S_t(M)."""
    Minv, det = inverse_and_jacobian(t)
    (a, b), (c, d) = Minv
    lo, hi = sorted(evolve(np.array([-1.0, 1.0]), 0.0, t)) # S_t(M) = [lo, hi]
    xs = 0.5 * (hi - lo) * xg + 0.5 * (hi + lo)
    jac = abs(det) / (c * xs + d) ** 2 # exp(Omega_t(x))
    return 0.5 * (hi - lo) * np.sum(wg * O(xs) * jac * 0.5)

# ------------------------------------------------------------------ fixed point
tau = np.tanh(T)
xstar = ((K - 1) - np.sqrt((K - 1) ** 2 + 4 * K * tau**2)) / (2 * K * tau)

# ------------------------------------------------------------------ non-uniform f0
# f0(x) = (3/4)(1 - x^2): Omega^{(0)} = -Lambda - G f0'/f0 = -2x - 2x = -4x
f0b = lambda x: 0.75 * (1 - x**2)
E0b = lambda vals: np.sum(wg * f0b(xg) * vals)
def direct_b(t):
    return E0b(O(evolve(xg, 0.0, t)))
def ttcf_b(t):
    n = int(np.floor(t / T + 1e-12))
    res = E0b(O(xg))
    for k in range(n + 1):
        a, b = k * T, min((k + 1) * T, t)
        if b - a < 1e-14:
            continue
        s_nodes = 0.5 * (b - a) * sg + 0.5 * (a + b)
        inner = np.array([E0b(-4 * xg * O(evolve(xg, s, t))) for s in s_nodes])
        res += 0.5 * (b - a) * np.sum(swg * inner)
    for k in range(1, n + 1):
```

```
        res += E0b(O(evolve(kick(xg), k * T, t)) - O(evolve(xg, k * T, t)))
    return res

if __name__ == "__main__":
    print(f"K = {K}, T = {T}, x* = {xstar:.10f}")
    print(f"{'t':>6} {'exact':>14} {'direct':>14} {'ttcf+jumps':>14} {'kawasaki':>14} support")
    for t in [0.25, 0.5, 0.75, 1.0, 1.3, 1.5, 2.0, 2.5, 3.0, 5.0]:
        lo, hi = sorted(evolve(np.array([-1.0, 1.0]), 0.0, t))
        print(f"{t:6.2f} {exact_mean_x(t):14.10f} {direct(t):14.10f} "
              f"{ttcf(t):14.10f} {kawasaki(t):14.10f} [{lo:.4f}, {hi:.4f}]")

    print("\nNon-uniform f0 = (3/4)(1-x^2):")
    print(f"{'t':>6} {'direct':>14} {'ttcf+jumps':>14}")
    for t in [0.25, 0.5, 1.0, 1.3, 2.0, 3.0]:
        print(f"{t:6.2f} {direct_b(t):14.10f} {ttcf_b(t):14.10f}")
```